Testing

05: DMA Control

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Let's talk P2

extend 1 week

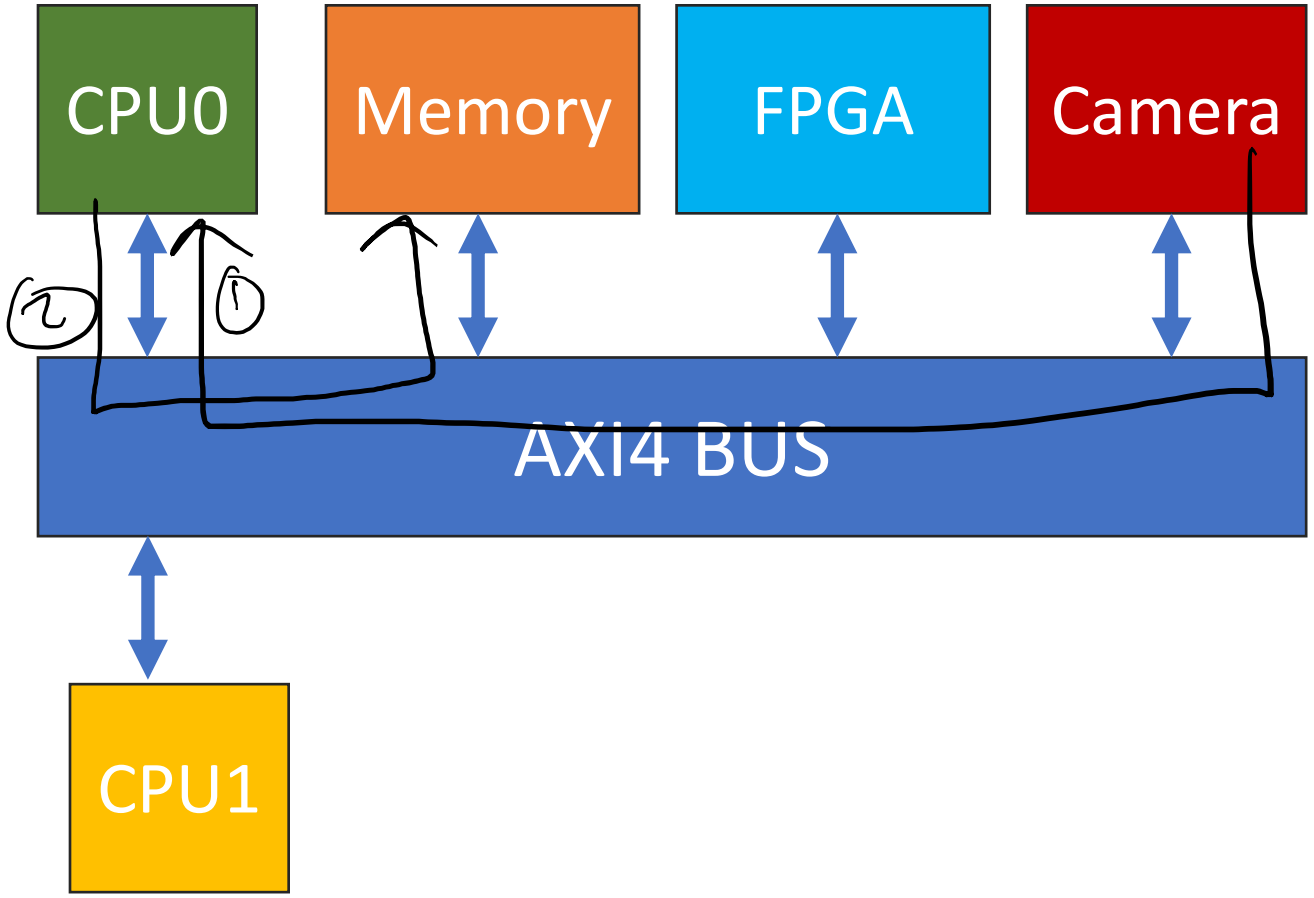
save some time @ ~5 to debug

Other Announcements

- Slack: Email Zach zwlevy@iu.edu
- Ethan's Office Hours schedule

Wednesday	5.30 – 8:00 pm
Thursday	2.30 – 4:00 pm

Q: How do I move data between the Camera and Memory?



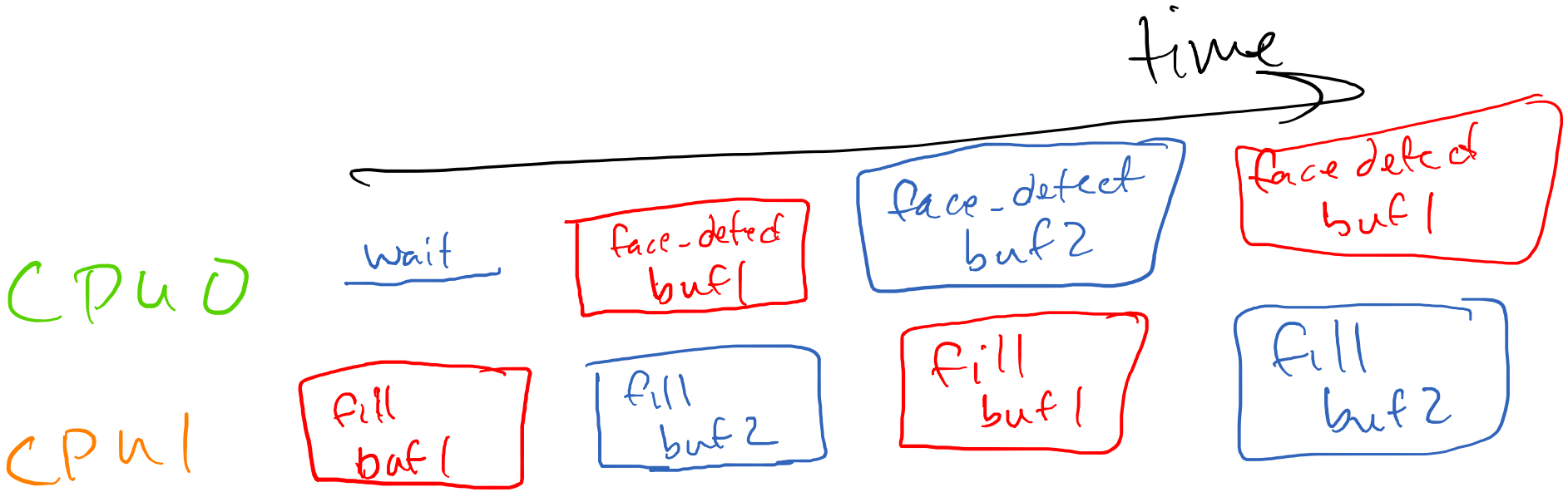
Parallel Execution using 2 CPUs

```
int main () {

    ask_cpu1_to_copy_image(camera, buf1, BUF_SIZE);
    wait_for_cpu1_done();

    while (true){
        ask_cpu1_to_copy_image(camera, buf2, BUF_SIZE);
        detect_face(buf1);
        wait_for_cpu1_done();

        ask_cpu1_to_copy_image(camera, buf1, BUF_SIZE);
        detect_face(buf2);
        wait_for_cpu1_done();
    }
}
```


time
CPU 0
CPU 1
wait
face-detect buf1
face-detect buf2
face detect buf1
fill buf1
fill buf2
fill buf1
fill buf2

DMA: Direct Memory Access

- A mini-CPU that does copy for you:

```
void copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

DMA

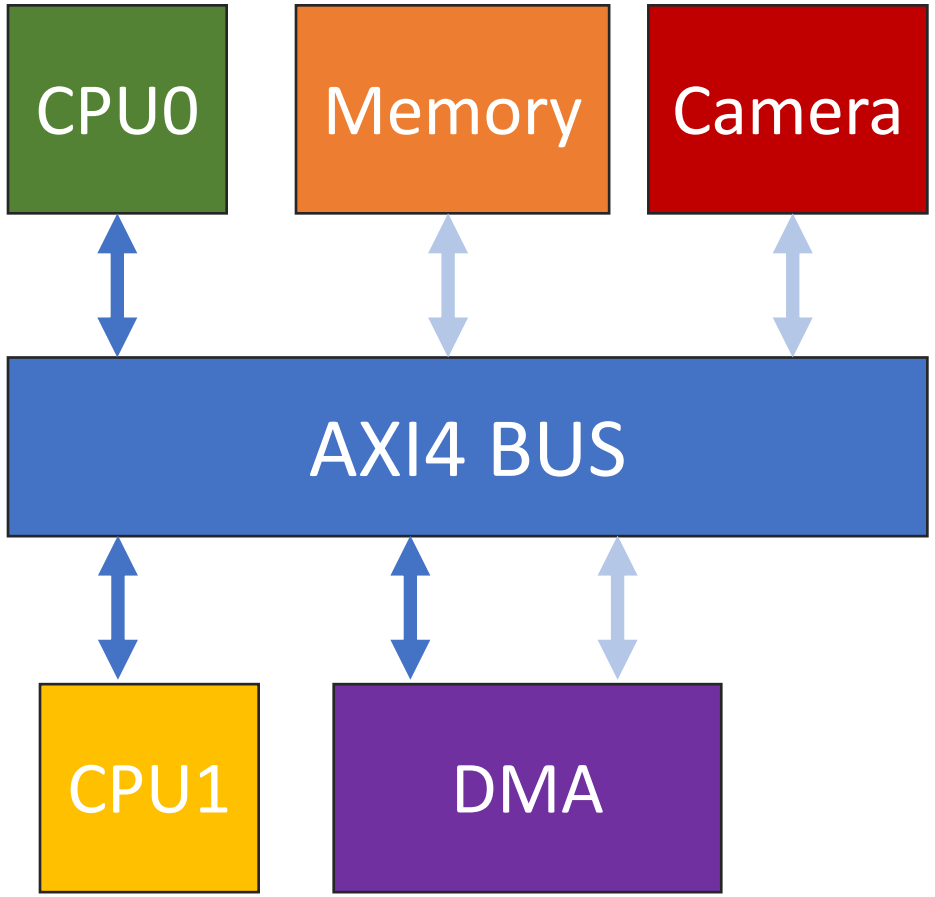
```
int main () {

    dma_copy_image(camera, buf1, BUF_SIZE);
    wait_for_dma_done();

    while (true){
        dma_copy_image(camera, buf2, BUF_SIZE);
        detect_face(buf1);
        wait_for_dma_done();

        dma_copy_image(camera, buf1, BUF_SIZE);
        detect_face(buf2);
        wait_for_dma_done();
    }
}
```

Why 2 interfaces?



- Interface 1: Memory Copy
  - Data Interface
  - Fast
  - Master
- Interface 2: Tell DMA <u>what</u> to copy
  - Control Interface
  - Slower
  - Slave

What do we need to do this?

```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
    register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;  //load

        to[i] = reg;  //store
    }
}
```

* loads + stores

// where from
source

// where to
destination

// how much
size

// start

// done

What do we need to do this?

```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

   for (int i = 0; i < size; ++i){

      reg = *from;

      to[i] = reg;
   }
}
```

- Ability to do loads
- Ability to do stores

- Start signal
- 3 32-bit inputs:
  - From, to, size
- Done signal

What do we need to do this?

```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

   for (int i = 0; i < size; ++i){

      reg = *from; //load

      to[i] = reg; //store
   }
}
```

- AXI4 Master Interface
  - Loads + Stores

AXI Slave

- 5 MMIO registers
  - Control (Start)
  - Status (Done)
  - Source (From)
  - Destination (To)
  - Size (in Bytes)

DMA V1.0 Interface

- 0x0400: Control Register (Start)
- 0x0404: Status Register (Done)
- 0x0408: Source Address
- 0x040C: Destination Address
- 0x0410: Transfer Size in Bytes

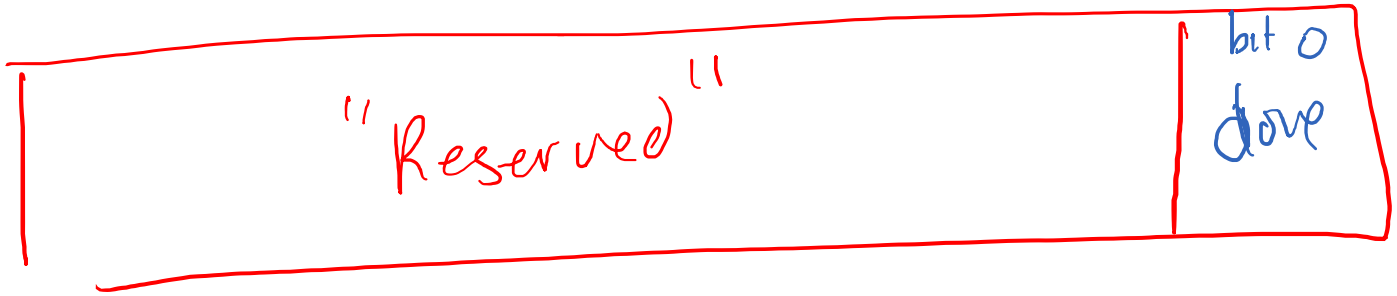
Control Register

0x0400 Control Register

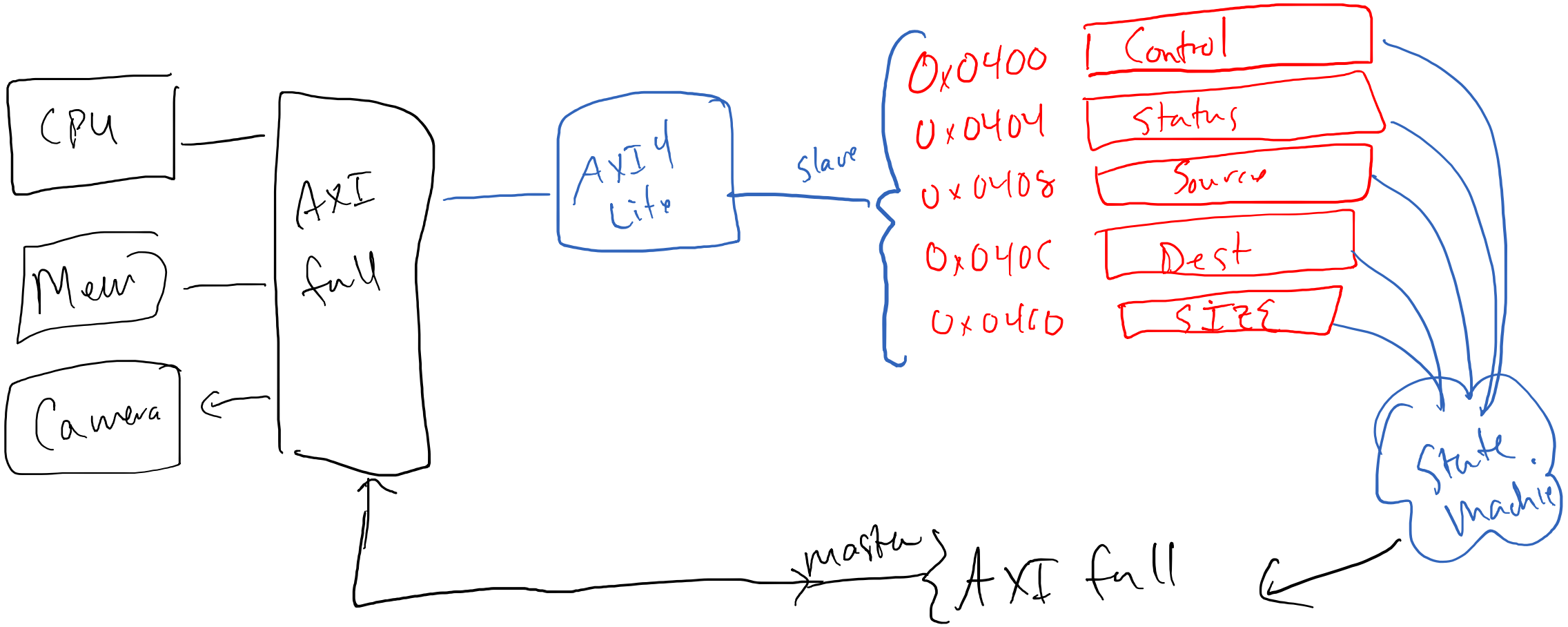
Status Register

0x0404 "Status" Reg

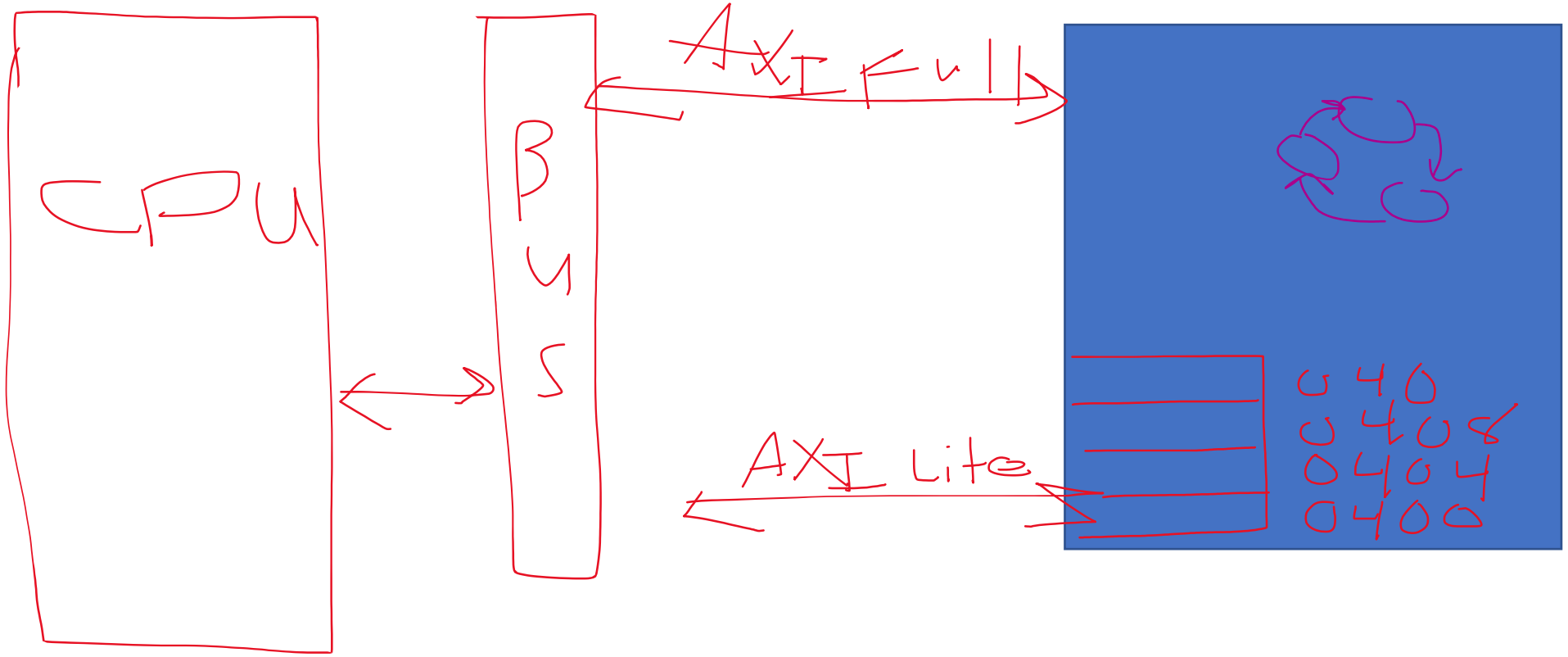
32-bits



DMA V1.0 Interface

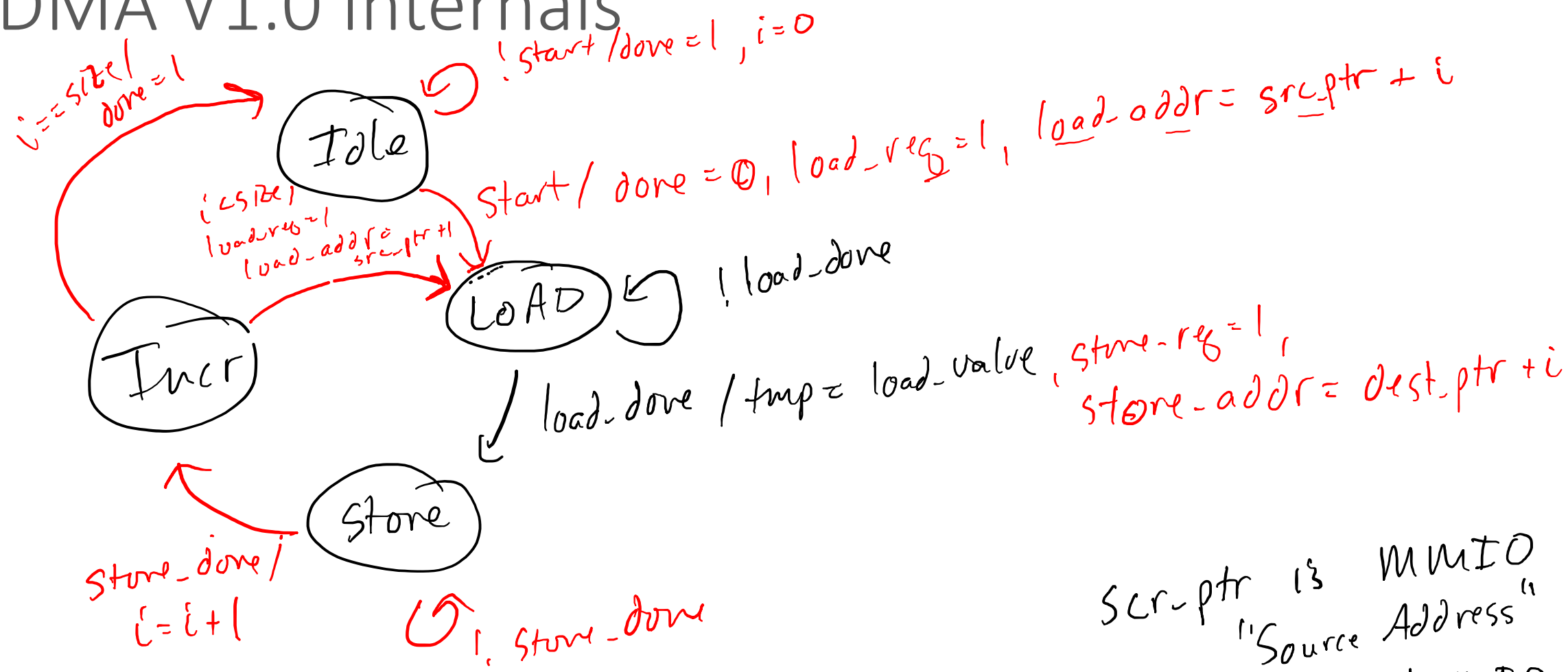


DMA V1.0 Interface



DMA V1.0 Internals

tmp = temporary value



scr_ptr is MMIO "Source Address"

dest_ptr is MMIO "Destination Address"

DMA V1.0 Internals

- IDLE: Status[Done]=1, wait for Control[Start]
- START: Status[Done] = 0, i = 0;
- LOAD: tmp = [Source+i]
- STORE: Dest+i = tmp
- BRANCH: i < TransferSize ? ++I, LOAD : IDLE

Does the AXI4 Full Interface have an address?

Does the AXI4 Full Interface have an MMIO Address?

- Is pretending to be memory, or a CPU?
- Does a CPU have a memory address?
- No.
- MMIO is for SLAVE interfaces.

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

```
void dma_copy ( uint32_t * from,// source (src)
                uint32_t * to, // destination (dest)
                uint32_t size){ (bytes)



    //code me!


}
```

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

```
void dma_copy ( uint32_t * from,  //source
                uint32_t * to,    //dest
                uint32_t size){

    *((uint32_t*)(0x0408)) = from;
    *((uint32_t*)(0x040C)) = to;
    *((uint32_t*)(0x0410)) = size;
    *((uint32_t*)(0x0400)) = 0x1; or 0xff;
    while ( *((uint32_t*)(0x0404)) != 0x1) {;}
}
```

* volatile missing

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

```
void dma_copy ( uint32_t * from,
                uint32_t * to,
                uint32_t size){

    *((uint32_t*)(0x0408))=from;
    *((uint32_t*)(0x040C))=to;
    *((uint32_t*)(0x0410))=size;
    *((uint32_t*)(0x0400))= 0x1; //start
    //spin until copy done?
    while( *((uint32_t*)(0x0404)) != 0x1){;}
}
```

`volatile` keyword omitted!

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

```
void dma_copy( uint32_t * from,
                                uint32_t * to,
                                uint32_t size){
        *((uint32_t)(0x0408))=from;
        *((uint32_t)(0x040C))=to;
        *((uint32_t)(0x0410))=size;
        *((uint32_t)(0x0400))= 0x1; //start
        }

void wait_for_dma(){
        //spin until copy done?
        while( *((uint32_t)(0x0404)) != 0x1){;}
}
```

volatile keyword omitted!

Real DMA

Register Address Map

Table 2-6: **AXI CDMA Register Summary**

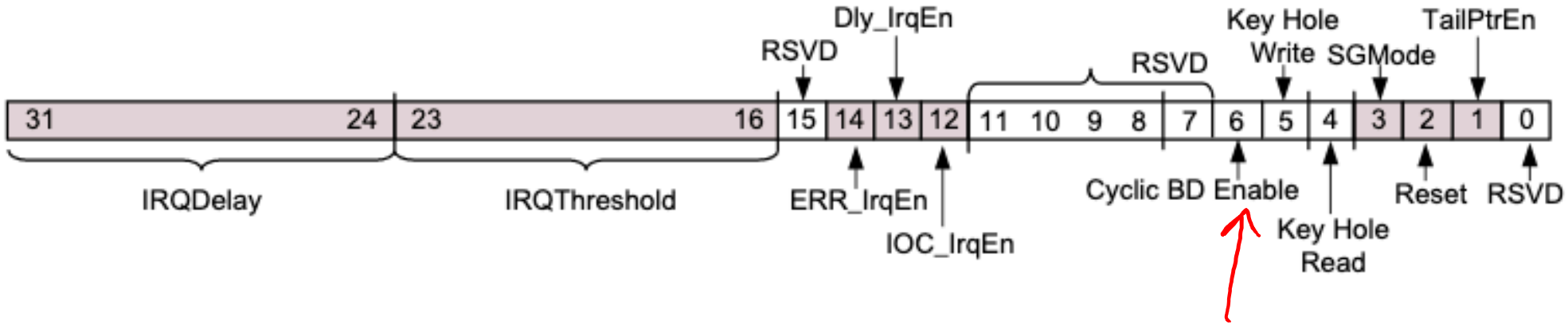
Address Space Offset[1]	Name	Description
00h	CDMACR	CDMA Control
04h	CDMASR	CDMA Status
08h	CURDESC_PNTR	Current Descriptor Pointer
0Ch[2]	CURDESC_PNTR_MSB	Current Descriptor Pointer. MSB 32 bits. Applicable only when the address space is greater than 32.
10h	TAILDESC_PNTR	Tail Descriptor Pointer
14h[2]	TAILDESC_PNTR_MSB	Tail Descriptor Pointer. MSB 32 bits. Applicable only when the address space is greater than 32. 64 bit
18h	SA	Source Address
1Ch[2]	SA_MSB	Source Address. MSB 32 bits. Applicable only when the address space is greater than 32. 64 bit
20h	DA	Destination Address
24h[2]	DA_MSB	Destination Address. MSB 32 bits. Applicable only when the address space is greater than 32. 64 bit
28h	BTT	Bytes to Transfer

Real DMA

Register Details

CDMACR (CDMA Control – Offset 00h)

This register provides software application control of the AXI CDMA.



Other DMA tweaks

```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size,
               uint32_t inc_from, uint32_t inc_to)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = (inc_from ? *from[i] : *from);

        if (inc_to) to[i] = reg;
        else        to = reg;
    }
}
```

Other DMA tweaks

- Interrupts (not in E315)
- Repeat the transfer?

Preview Project 3

0x0001 → 0xfffe

References

- https://www.xilinx.com/support/documentation/ip_documentation/axi_cdma/v4_1/pg034-axi-cdma.pdf

05: DMA Control

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University